Daws86s-Interview Preparation Series(DevSecOps Linux-Day2)

1. What happens when you type a command in Linux?

> The shell finds the command, the kernel runs it, and the shell gets the result backThe kernel then executes it using CPU, memory, and other resources, and when it finishes, an exit status is returned to the shell."

2. Difference between Linux kernel and shell

> "The kernel is the core of Linux — it talks directly to the hardware and manages things like CPU scheduling, memory, disk, and networking.
> The shell is just an interface. It's how we tell the kernel what we want to do.
> So basically, the kernel does the actual work, the shell is how we communicate with it."

3. What is init? Explain run levels / systemd targets

> **Init** is the first user-space process started by the kernel (PID 1) that controls system startup by bringing the machine into a defined operating state using **runlevels or systemd targets**.

4. Difference between hard link and soft link

> "A hard link points directly to the same inode as the original file, so even if the original file is deleted, the data still exists.
> A soft link is just a shortcut — if the original file is removed, the link breaks.
> In real life, we mostly use soft links for configs and versioned binaries."

Create hard link

```
ln original.txt hardlink.txt
```

Create soft (symbolic) link

```
ln -s original.txt softlink.txt
```

Check inode numbers

```
ls -li
```

Remove original file (test behavior)

```
rm original.txt
```

5. What is an inode?

> "An inode is where Linux stores metadata about a file — permissions, ownership, size, and disk blocks.
> The file name doesn't matter; it just points to the inode.
> That's why you can run out of inodes even when disk space is still available."

```
ls -li filename
```

6. Explain Linux file permissions and ownership

> "Linux controls access using permissions — read, write, and execute — applied to the owner, group, and others.
> Ownership decides who can do what.
> Most production issues I've seen around permissions are either wrong ownership or overly open permissions."

In Linux, every file or directory has an **owner**, a **group**, and **permissions** that decide who can read, write, or execute it.

First, you **check permissions and ownership**:

```
ls -l
```

You'll see something like:

```
-rwxr-xr-- 1 user group file.sh
```

- `r` = read, `w` = write, `x` = execute
- First 3 → owner, next 3 → group, last 3 → others
- `-` file, `d` directory, l link

If you want **inode-level details**:

```
stat file.sh
```

To **change permissions**:

```
chmod 755 file.sh     # numeric (most used)
chmod u+x file.sh     # symbolic
```

To **change ownership**:

```
chown user file.sh
chown user:group file.sh
chown -R user:group /path
```

To **change only group**:

```
chgrp group file.sh
```

To **check user and group info**:

```
whoami
id
groups
```

If a script fails with **permission denied**:

```
chmod +x script.sh
./script.sh
```

For **directories**:

- `r` → list files
- `w` → create/delete
- `x` → enter (cd)

```
chmod 755 dir
```

Linux permissions control file access for owner, group, and others, and most real-world issues are solved using `ls -l`, `chmod`, and `chown`.

7. What is umask?

> "Umask defines the default permissions for newly created files and directories.
> It's basically a safety net to make sure files aren't created with overly permissive access.
> In servers, umask is important for security."

8. What is swap memory and when is it used?

> "Swap is disk space used as a backup for RAM.
> When memory usage spikes, Linux moves less-used pages to swap to avoid crashing.
> If a system is constantly using swap, it usually means the server is under-provisioned."

Swap Partition

```
lsblk
```

Look for type:

```
swap
```

Swap File (Most common in cloud)

```
ls -lh /swapfile
```

I create a swap file and enable it temporarily or permanently based on workload needs.

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
```

10. Difference between process and thread

> "A process has its own memory space, while threads share memory within the same process.
> Threads are lighter and faster, but processes are more isolated.
> Web servers typically use threads, while services are separated as processes."

11. What is a thread?

> "A thread is the smallest unit of execution inside a process.
> Multiple threads allow a program to do multiple things at the same time without starting multiple processes."

12. What is a daemon process?

> "A daemon is a background process that usually starts at boot and keeps running.
> Examples are sshd, nginx, cron.
> You normally don't interact with it directly, but your system depends on it."

13. Difference between ps, top, and htop

> "ps gives a snapshot of processes at a point in time.
> top gives a real-time view.
> htop is like top but more interactive and easier to use — that's what I prefer on servers."

14. What is load average?

> "Load average shows how many processes are waiting for CPU over time — 1, 5, and 15 minutes.
> If the load is consistently higher than the number of CPU cores, the system is under stress."

15. What happens when Linux runs out of memory?

> "Linux first tries to use swap.
> If that's not enough, the OOM killer steps in and kills processes to protect the system.
> This is very common in containerized environments if memory limits are wrong

16. What is OOM killer?

> "It's a kernel mechanism that kills processes when the system runs out of memory.
> It chooses processes based on memory usage and priority."

17. What is ulimit?

> "ulimit controls how many resources a user or process can consume — like open files or processes.
> It's critical for high-traffic applications to avoid hitting file descriptor limits."

18. What is a zombie process?

> "A zombie process has finished execution but still exists because its parent hasn't collected its exit status.
> It doesn't consume resources, but too many zombies indicate a badly written parent process."

19. What is an orphan process?

> "If a parent process dies, its child becomes an orphan and gets adopted by systemd or init.
> That's normal behavior in Linux."

20. Difference between user space and kernel space

> "User space is where applications run.
> Kernel space is where the kernel runs and interacts with hardware.
> This separation keeps the system stable and secure."

21. How do you check running processes in Linux?

> "Usually I start with `top` or `htop` to get a real-time view.
> If I need a quick snapshot or want to filter something, I use `ps aux` or `ps -ef`.
> For troubleshooting, `htop` is my first choice because it's interactive."

22. What is nice and renice?

> "Nice and renice control a process's CPU priority.
> A higher nice value means lower priority.
> I use `renice` when a background job is consuming too much CPU and affecting critical services."

23. How do you kill a process?

> "I always try a graceful stop first using
> If the process doesn't respond, then I use force kill as a last option.

24. What is load average?

> "Load average shows how many processes are waiting to run on the CPU.
> If the load is higher than the number of CPU cores for a long time, the system will feel slow."

25. How do you troubleshoot high CPU usage?

> "First I check `top` or `htop` to see which process is consuming CPU.
> Then I check whether it's expected behavior or a bug.
> If needed, I check logs, adjust limits, or restart the service."

26. How do you troubleshoot high memory usage?

> "I check memory usage using `free -m` and `top`.
> If a process keeps growing, it might be a memory leak.
> In production, this usually means tuning the app or increasing memory."

27. How do you check disk usage and find large files?

> "I use `df -h` to check overall disk usage.
> If a disk is filling up, I use `du -sh` and `find` to locate large directories or files."

28. What happens when disk is 100% full?

> "When disk is full, applications can't write logs or temp files.
> Services may crash, SSH may stop working, and the system becomes unstable.
> Disk full issues are very common in production outages."

29. What do you do when disk is full in production?

> "First I identify what's consuming space — usually logs.
> Then I clean up safely, rotate logs, or move data.
> Long term, I fix log rotation or increase disk size."

30. What is swap usage and how do you monitor it?

> "I check swap usage using `free` or `vmstat`.
> If swap is constantly being used, it's a sign of memory pressure.
> Occasional swap usage is fine, but constant swapping means performance issues."

31. What is a zombie process and how do you handle it?

> "A zombie process means the parent process didn't clean up properly.
> You can't kill a zombie directly — you have to restart or fix the parent process."

32. Difference between zombie and orphan process

> "Zombie processes are already dead but not cleaned up.
> Orphan processes are still running and get adopted by systemd.
> Zombies are bad coding behavior; orphans are normal."

33. How do you check which process is using a specific port?

> "I usually use `ss -tulnp` or `lsof -i :PORT`.
> This is very useful when a service fails to start due to port conflicts."

34. What is CPU steal time?

> "Steal time happens mostly in virtualized environments when the hypervisor takes CPU away from your VM.
> High steal time usually means the host is overcommitted."

35. How do you check memory leaks in Linux?

> "At OS level, I watch a process's memory over time using `top` or `ps`.
> If memory keeps increasing and never drops, it's likely a leak and needs application-level debugging."

36. What is iowait?

> "Iowait means the CPU is idle but waiting for disk operations.
> High iowait usually indicates slow disks or heavy I/O operations."

37. How do you troubleshoot high I/O wait?

> "I check disk usage using `iostat` or `iotop`.
> Then I see which process is causing heavy disk reads or writes."

38. Difference between vmstat and iostat

> "vmstat gives an overview of CPU, memory, and processes.
> iostat focuses more on disk performance.
> I usually start with vmstat and then drill down with iostat."

39. What is context switching?

> "Context switching happens when the CPU switches between processes or threads.
> Too many context switches usually mean the system is overloaded."

40. How do you identify a slow Linux system?

> "I check CPU, memory, disk I/O, and load average.
> If all look fine, I check network latency or application logs.
> Slowness is rarely just one metric — it's usually a combination."

41. What is a filesystem?

> "A filesystem is how Linux organizes and stores data on disk.
> It defines how files are created, stored, accessed, and deleted.
> Without a filesystem, the OS can't really use the disk properly."

42. Difference between ext4 and xfs (real-world view)

> "Ext4 is very stable and works well for most workloads.
> XFS performs better with large files and heavy I/O, like logs or databases.
> In production, OS disks are usually ext4, and data disks are often XFS."

43. What is df vs du?

> "`df` shows disk usage at filesystem level.
> `du` shows disk usage at directory or file level.
> If df shows disk full, du helps me find *what* is consuming space."

44. Disk is full but df -h shows free space — why?

> "This usually happens due to deleted files still being held open by a process.
> The space won't be released until that process is restarted.
> I use `lsof | grep deleted` to confirm this."

45. What are inodes and inode exhaustion?

> "Each file consumes an inode.
> You can run out of inodes even if disk space is available — this happens when there are too many small files.
> I've seen this often with log files or temp directories."

46. How do you check inode usage?

> "I use `df -i`.
> If inode usage is 100%, applications will fail even though disk space looks fine."

47. What is mount in Linux?

> "Mounting is attaching a filesystem to a directory so Linux can access it.
> Without mounting, the disk exists but isn't usable."

48. What is /etc/fstab?

> "`/etc/fstab` defines permanent mounts.
> Anything listed there is mounted automatically at boot.
> A wrong fstab entry can cause boot failures, so I'm always careful editing it."

49. Temporary vs permanent mount

> "Temporary mounts are done using the `mount` command and disappear after reboot.
> Permanent mounts are defined in `/etc/fstab` and survive reboots."

50. What is LVM and why is it used?

> "LVM allows flexible disk management.
> You can resize volumes, add disks, and take snapshots without downtime.
> It's extremely useful in production environments."

51. How do you extend disk space without downtime?

> "If the disk uses LVM, I extend the volume and then resize the filesystem.
> This can usually be done online without restarting services."

52. Difference between block storage and file storage

> "Block storage is raw disk attached to a server, like EBS.
> File storage provides a shared filesystem, like NFS.
> Databases prefer block storage; shared data prefers file storage."

53. What is RAID?

> "RAID combines multiple disks for performance or redundancy.
> Different RAID levels serve different purposes — speed, fault tolerance, or both."

54. What is swap partition vs swap file?

> "Swap partition is a dedicated disk partition.
> Swap file is a regular file used as swap.
> Swap files are easier to resize and manage."

55. How do you troubleshoot disk I/O issues?

> "I check disk usage and I/O wait using `iostat` and `iotop`.
> Then I identify which process is causing heavy disk operations."

56. What is fsck?

> "fsck checks and repairs filesystem inconsistencies.
> It's usually run at boot or manually when disk corruption is suspected."

57. What happens if /root or /var fills up?

> "If `/var` fills up, logs stop working and services fail.
> If `/root` fills up, administrative tasks become difficult.
> Most disk issues I've seen involve `/var`."

58. How do you manage log growth?

> "I use log rotation with logrotate.
> This prevents logs from growing indefinitely and filling the disk."

59. What is logrotate?

> "logrotate rotates, compresses, and deletes old logs automatically.
> It's critical for long-running servers."

60. How do you prevent disk issues in production?

> "I monitor disk usage, configure log rotation, and set alerts.
> Disk problems are predictable — monitoring prevents outages."

61. How do you check open ports in Linux?

> "I usually use `ss -tulnp`.
> It's fast and gives me which service is listening on which port.
> If I want more details about the process, I use `lsof -i`."

62. Difference between netstat, ss, and lsof

> "`netstat` is old and mostly deprecated.
> `ss` is the modern replacement and much faster.
> `lsof` is useful when I want to see which process is using a port or a network connection."

63. How do you troubleshoot network connectivity issues?

> "First, I check IP configuration.
> Then I test connectivity using ping.
> If ping works but the app doesn't, I check ports, firewall rules, and application logs."

64. How does DNS resolution work in Linux?

> "Linux checks `/etc/nsswitch.conf` and `/etc/resolv.conf`.
> It then queries the configured DNS server.
> If DNS is slow or broken, applications appear to hang even though the network is up."

65. Difference between ping, traceroute, and curl

> "Ping checks reachability.
> Traceroute shows the network path.
> Curl tests application-level connectivity, which is what really matters in production."

66. How do you check which DNS server is being used?

> "I check `/etc/resolv.conf`.
> That tells me which nameservers the system is querying."

67. How do you debug DNS issues?

> "I use `nslookup` or `dig` to test name resolution.
> If that fails, I check DNS server reachability and firewall rules."

68. What is TCP vs UDP (OS level)?

> "TCP is reliable and connection-based.
> UDP is faster but doesn't guarantee delivery.
> Web apps mostly use TCP, while DNS and streaming often use UDP."

69. What is a port?

> "A port identifies a service on a machine.
> IP tells you the server, port tells you the service."

70. What is a socket?

> "A socket is a combination of IP, port, and protocol.
> It's how applications communicate over the network."

71. How do you check active connections?

> "I use `ss -an` or `netstat -an`.
> This helps during traffic spikes or DDoS situations."

72. What is localhost (127.0.0.1)?

> "It's the loopback address.
> Traffic sent there never leaves the machine and is used for local communication."

73. What is a firewall in Linux?

> "A firewall controls which traffic is allowed in or out.
> In Linux, this is usually handled by iptables, nftables, or firewalld."

74. How do you check firewall rules?

> "Depends on the setup.
> For iptables, I check `iptables -L`.
> For firewalld, I use `firewall-cmd --list-all`."

75. What is iptables / nftables used for?

> "They're used to filter and control network traffic.
> In production, firewall misconfiguration is a common cause of downtime."

76. How do you troubleshoot a service that is not accessible over network?

> "I check if the service is running, then if it's listening on the correct port.
> After that, I check firewall rules and network routes."

77. What is NAT?

> "NAT translates private IPs to public IPs.
> It's commonly used in cloud environments."

78. What is a proxy?

> "A proxy sits between clients and servers.
> It's used for security, caching, or access control."

79. How do you configure proxy in Linux?

> "Usually by setting environment variables like `http_proxy` and `https_proxy`.
> For system-wide use, I configure it in profile or service files."

80. Common networking mistakes in production

> "Wrong firewall rules, wrong DNS, wrong ports, and assuming network is the problem when it's actually the application."

81. Where are Linux logs stored?

> "Most Linux logs are under `/var/log`.
> Depending on the distro, system logs might be in `syslog` or `messages`.
> Application-specific logs usually have their own directories."

82. How do you troubleshoot a service that is not starting?

> "First I check the service status using `systemctl status`.
> If that doesn't give enough info, I check logs using `journalctl`.
> Most startup issues are configuration or permission related."

83. Difference between journalctl and traditional logs

> "Traditional logs are plain text files.
> journalctl stores logs in a structured, indexed format.
> journalctl is much easier to filter and search during incidents."

84. How do you debug a Linux service crash?

> "I check logs around the crash time.
> Then I look at system resources like memory or disk.
> If it keeps crashing, I check configuration changes or recent deployments."

85. How do you check boot logs?

> "I use `journalctl -b`.
> It shows logs from the current boot, which is very useful after unexpected reboots."

86. How do you check logs for a specific service?

> "With systemd, I use `journalctl -u service-name`.
> It's faster than hunting through multiple log files."

87. What is dmesg?

> "dmesg shows kernel-related messages.
> It's useful for hardware issues, disk errors, or kernel panics."

88. What do you check when a system reboots unexpectedly?

> "I check boot logs, kernel messages, and system logs.
> Common causes are kernel panic, OOM killer, or hardware issues."

89. Logs are not getting generated — what do you check?

> "I check disk space and permissions first.
> Then I check if the logging service is running.
> Most of the time, it's either disk full or wrong permissions."

90. How do you monitor logs in real time?

> "I use `tail -f` for quick checks.
> For production, logs are usually shipped to centralized logging systems."

91. What is log rotation and why is it important?

> "Log rotation prevents logs from growing endlessly.
> Without it, disks fill up and services crash."

92. What tool is commonly used for log rotation?

> "logrotate.
> It rotates, compresses, and removes old logs automatically."

93. How do you troubleshoot application errors using logs?

> "I look for error patterns around the failure time.
> I also check timestamps to correlate logs with deployments or traffic spikes."

94. Difference between error logs and access logs

> "Access logs show requests.
> Error logs show failures.
> Both are important, but error logs are usually my first stop."

95. What are common log-related production issues?

> "Disk full, missing logs, wrong permissions, and logs being overwritten."

96. How do you search logs efficiently?

> "I use grep with timestamps or keywords.
> For large logs, I combine grep with awk or less."

97. What is centralized logging?

> "Centralized logging collects logs from multiple servers into one place.
> It's essential when you're managing large-scale systems."

98. How do you debug performance issues using logs?

> "I correlate slow response times with logs.
> Often the issue becomes clear when you match logs with traffic or errors."

99. How do you handle sensitive data in logs?

> "Sensitive data should never be logged.
> If unavoidable, logs should be masked and access restricted."

100. Best practice for log management in production

> "Rotate logs, monitor disk usage, centralize logs, and set alerts.
> Logs are useless if you can't access them during an incident."

101. Difference between `sudo` and `su`

> “`su` switches you completely to another user, usually root.
> `sudo` lets you run specific commands with elevated privileges.
> In production, `sudo` is preferred because it’s auditable and safer.”

102. Why is `sudo` preferred in production?

> “Because it provides control and traceability.
> You can limit what commands a user can run, and everything is logged.
> With `su`, once you’re root, there’s no control.”

103. What is `/etc/sudoers`?

> “It’s the configuration file that defines who can run what using sudo.
> I always edit it using `visudo` to avoid syntax errors.”

104. What is SELinux?

> “SELinux is an additional security layer that enforces mandatory access control.
> Even if permissions look correct, SELinux can still block access.
> It’s powerful but often confusing when you first encounter it.”

105. Why do DevOps engineers fear SELinux?

> “Because services fail silently if policies aren’t correct.
> If you don’t understand SELinux, debugging becomes painful.
> But once you know how to read SELinux logs, it’s manageable.”

106. SELinux modes

> “Enforcing blocks unauthorized actions.
> Permissive logs violations but doesn’t block.
> Disabled turns it off completely.”

107. How do you troubleshoot SELinux issues?

> “First I check if SELinux is enforcing.
> Then I look at audit logs.
> If needed, I temporarily switch to permissive mode to confirm the issue.”

108. What is SSH?

> “SSH is a secure way to access a remote Linux system.
> It encrypts communication and is the standard for server access.”

109. How does SSH key-based authentication work?

> “The client has a private key and the server stores the public key.
> During login, the server verifies the client without needing a password.
> It’s much more secure than password-based login.”

110. Why should password login be disabled?

> “Passwords can be brute-forced.
> SSH keys are much harder to break.
> In production, disabling password login is a basic security practice.”

111. Where are SSH keys stored?

> “On the client side, private keys are stored locally.
> On the server, public keys are stored in `~/.ssh/authorized_keys`.”

112. What is `known_hosts`?

> “`known_hosts` stores fingerprints of servers you’ve connected to.
> It prevents man-in-the-middle attacks by warning if a server’s identity changes.”

113. Permission denied (publickey) — how do you troubleshoot?

> “First I check if the correct key is being used.
> Then I verify permissions on `.ssh` and `authorized_keys`.
> Most issues are permission-related.”

114. Correct permissions for SSH files

> “`.ssh` directory should be `700`.
> `authorized_keys` should be `600`.
> Wrong permissions will cause SSH to fail.”

115. How do you secure a Linux server?

> “I disable root login, use SSH keys, configure a firewall, keep the system updated, and monitor logs.
> Security is a continuous process, not a one-time setup.”

116. What is root login and why disable it?

> “Root login gives full access.
> Disabling it forces users to authenticate first and then escalate privileges, which is safer.”

117. How do you audit user activity in Linux?

> “I rely on sudo logs, SSH logs, and sometimes auditd.
> Auditing is important for compliance and security incidents.”

118. What is `fail2ban`?

> “fail2ban blocks IPs after repeated failed login attempts.
> It’s very effective against brute-force attacks.”

119. What are common security mistakes in Linux servers?

> “Using password login, open ports, outdated packages, and running everything as root.”

120. Final security best practice mindset

> “Security should be layered — OS hardening, network controls, monitoring, and audits.
> No single tool makes a system secure.”

121. What is automation in Linux from a DevOps point of view?

> “Automation in Linux means reducing manual work using scripts or tools.
> Anything repetitive — backups, monitoring, cleanup — should be automated.
> Manual work doesn’t scale and always leads to mistakes.”

122. What is a cron job?

> “Cron is a scheduler in Linux.
> It’s used to run commands or scripts automatically at a specific time or interval.
> Most routine maintenance tasks use cron.”

123. Where are cron jobs defined?

> “User-specific cron jobs are defined using `crontab -e`.
> System-wide cron jobs are usually under `/etc/crontab` or `/etc/cron.*` directories.”

124. Explain crontab format

> “A cron entry has time fields followed by the command.
> Minute, hour, day, month, and weekday.
> Once you understand the pattern, it’s very easy to use.”

125. How do you check existing cron jobs?

> “I use `crontab -l` for the current user.
> For system jobs, I check `/etc/crontab`.”

126. Common issues with cron jobs not running

> “Wrong paths, missing permissions, environment variables not available, or scripts not executable.
> Cron runs in a minimal environment, which surprises many people.”

127. How do you debug a failing cron job?

> “I redirect output and errors to a log file.
> Then I run the script manually using the same user to reproduce the issue.”

128. Difference between cron and systemd timers

> “Cron is simple and old.
> systemd timers are more flexible, have better logging, and handle dependencies.
> For modern systems, systemd timers are often preferred.”

129. What is a shell script?

> “A shell script is a file containing Linux commands that run in sequence.
> It’s used to automate tasks.”

130. What is a shebang (`#!/bin/bash`)?

> “The shebang tells the system which interpreter should run the script.
> Without it, the script may not run correctly.”

131. How do you create and run a shell script?

> “I create a file, add commands, add the shebang, make it executable using `chmod +x`, and then run it.”

132. How do you make a script executable?

> “Using `chmod +x script.sh`.
> Without execute permission, the script won’t run.”

133. How do you pass arguments to a script?

> “Arguments are passed after the script name and accessed using `$1`, `$2`, and so on.”

134. What are positional parameters?

> “They are variables like `$1`, `$2`, `$@`, and `$#` that represent script arguments.”

135. Difference between `$@` and `$*`

> “`$@` treats each argument separately.
> `$*` combines all arguments into a single string.
> I almost always use `$@`.”

136. How do you take user input in a script?

> “Using the `read` command.
> It’s useful for interactive scripts, but not recommended for automation jobs.”

137. What are variables in Bash?

> “Variables store values like paths, usernames, or counts.
> They make scripts flexible and reusable.”

138. Difference between local and environment variables

> “Local variables exist only in the script.
> Environment variables are available to child processes.”

139. How do you redirect output in Linux?

> “`>` overwrites output, `>>` appends.
> `2>` redirects errors.
> Redirection is very useful in cron jobs.”

140. What is piping (`|`) and why is it important?

> “Piping passes output of one command to another.
> It allows chaining commands and makes Linux very powerful.”

141. Difference between `#!/bin/bash` and `#!/bin/sh`

> “`/bin/bash` gives you full Bash features like arrays and advanced syntax.
> `/bin/sh` is more basic and meant to be portable.
> In most DevOps scripts, I explicitly use bash to avoid surprises.”

142. How does Bash interpret a script internally?

> “Bash reads the script line by line, expands variables, evaluates conditions, and executes commands in order.
> Before execution, it performs things like variable expansion and command substitution.”

143. Difference between sourcing a script and executing it

> “If I execute a script, it runs in a new shell.
> If I source it, it runs in the current shell.
> Sourcing is useful when I want variables or functions to remain available.”

144. What are positional parameters?

> “They are arguments passed to a script or function — `$1`, `$2`, `$@`, etc.
> They allow scripts to behave dynamically.”

145. `$@` vs `$*` — difference (practical)

> “`$@` preserves arguments individually, which is safer.
> `$*` merges everything into one string.
> I almost always use `$@`.”

146. How does variable scope work in Bash?

> “By default, variables are global.
> Inside functions, I use `local` to limit scope.
> Without local variables, scripts become hard to debug.”

147. `[[ ]]` vs `[ ]`

> “`[[ ]]` is more powerful and safer.
> It handles strings and patterns better.
> I prefer `[[ ]]` whenever I write Bash scripts.”

148. Case statement vs if-else — when to use?

> “If I have many conditions, I use `case` — it’s cleaner and easier to read.
> For simple checks, `if-else` is enough.”

149. For loop vs while loop — real use cases

> “I use `for` loops when iterating over a known list.
> I use `while` loops when looping until a condition is met.”

150. How do you break nested loops?

> “I use `break` with a number, like `break 2`.
> That exits multiple loop levels.”

151. What happens if a condition fails in a script?

> “Normally, the script continues.
> That’s why error handling is important.
> With `set -e`, the script exits immediately on failure.”

152. How do you define and use functions in Bash?

> “Functions are blocks of reusable code.
> They help avoid repetition and make scripts cleaner.”

153. How do you pass arguments to a function?

> “Same way as scripts — using `$1`, `$2`, etc.
> Functions in Bash behave like mini-scripts.”

154. Return value vs exit code — difference

> “Return values are for functions.
> Exit codes are for scripts.
> Both use numeric values, but they serve different purposes.”

155. How do you reuse scripts across projects?

> “I keep common scripts in a shared repo or directory.
> Then I source them or call them as needed.
> Reusability saves time and reduces bugs.”

156. How do you debug function-level issues?

> “I add echo statements or enable debug mode using `set -x`.
> This helps me see exactly what’s being executed.”

157. What is `set -e`?

> “It makes the script exit immediately if any command fails.
> It prevents silent failures, which are dangerous in automation.”

158. What is `set -u`?

> “It fails the script if an undefined variable is used.
> Very useful to catch typos early.”

159. What is `set -o pipefail`?

> “It ensures a pipeline fails if any command in it fails.
> Without this, failures can go unnoticed.”

160. How do you handle failures gracefully in scripts?

> “I check exit codes, log errors, and sometimes use retries.
> Automation should fail clearly, not silently.”

161. How do you log errors in Bash scripts?

> “I usually redirect errors to a log file using `2>>`.
> For important scripts, I explicitly log timestamps and error messages so I can trace issues later.”

162. How do you debug a script running in production?

> “First, I don’t panic and don’t modify it blindly.
> I enable debug mode with `set -x` in a controlled way or add temporary logging.
> I always try to reproduce the issue in a lower environment if possible.”

163. Script works locally but fails in CI — why?

> “Most of the time it’s environment-related.
> Different PATH, missing tools, permissions, or environment variables.
> CI environments are minimal, so assumptions break.”

164. Script stuck in an infinite loop — how do you debug?

> “I add logging inside the loop to see what condition isn’t changing.
> In many cases, a variable isn’t getting updated as expected.”

165. Script running slow — how do you optimize?

> “I check if I’m calling external commands inside loops.
> Replacing multiple calls with a single command usually improves performance a lot.”

166. How do you handle secrets securely in Bash scripts?

> “I never hardcode secrets.
> I use environment variables, secret managers, or encrypted files.
> If secrets appear in scripts, it’s already a security issue.”

167. How do you avoid hardcoding credentials?

> “By injecting them at runtime using environment variables or CI/CD secret stores.
> This is non-negotiable in production.”

168. What is `trap` in Bash?

> “`trap` lets me run cleanup code when a script exits or receives a signal.
> It’s useful for removing temp files or unlocking resources.”

169. Difference between exit code 0 and non-zero

> “Exit code 0 means success.
> Non-zero means failure.
> Automation tools rely heavily on exit codes.”

170. What does `$?` represent?

> “It shows the exit code of the last executed command.
> I use it often in conditional checks.”

171. How do you retry commands in a script?

> “I usually wrap the command in a loop with a retry counter.
> Retries are useful for network-related failures.”

172. How do you ensure scripts fail fast?

> “I use `set -e`, `set -u`, and `set -o pipefail`.
> Failing fast is better than silent corruption.”

173. How do you clean up temporary files?

> “I create temp files in `/tmp` and remove them using `trap`.
> Leaving temp files behind causes disk issues over time.”

174. How do you monitor scripts?

> “I log output, monitor exit codes, and integrate with monitoring or alerting tools.
> Silent scripts are dangerous.”

175. How do you write scripts that are safe to rerun?

> “I make scripts idempotent.
> Running them multiple times should not break anything.”

176. How do you write a disk monitoring script?

> “I check disk usage using `df`, compare against a threshold, and trigger alerts.
> This is a very common interview question.”

177. Script to check if a service is running

> “I usually check using `systemctl is-active`.
> If it’s not running, I log and alert.”

178. Script to restart a service if it is down

> “I check service status and restart only if it’s inactive.
> Blind restarts can hide deeper problems.”

179. How do you automate log cleanup?

> “Using logrotate or a scheduled cleanup script.
> Manual log cleanup doesn’t scale.”

180. When should Bash NOT be used and Python preferred?

> “If logic becomes complex, involves APIs, or heavy data processing, Python is better.
> Bash is great for orchestration, not complex logic.”

181. Explain the Linux boot process in detail

> “When a Linux system starts, the firmware (BIOS or UEFI) initializes the hardware.
> Then the bootloader loads the Linux kernel into memory.
> The kernel initializes hardware, mounts the root filesystem, and then starts systemd.
> From there, systemd brings up all required services.”

182. Difference between BIOS and UEFI

> “BIOS is the older firmware system and has limitations like disk size.
> UEFI is modern, faster, and supports features like secure boot.
> Most modern servers use UEFI.”

183. What happens after the kernel is loaded?

> “The kernel initializes devices, memory, and drivers.
> Then it mounts the root filesystem and starts the first user-space process, which is systemd.”

184. What is systemd and why is it important?

> “systemd is the init system in modern Linux.
> It manages services, startup order, logging, and restarts.
> It made Linux boot faster and service management easier.”

185. What is PID 1 and why is it special?

> “PID 1 is the first process started by the kernel.
> It’s responsible for managing all other processes.
> If PID 1 crashes, the system usually goes down.”

186. What is user space vs kernel space?

> “User space is where applications run.
> Kernel space is where the kernel operates and interacts with hardware.
> This separation protects system stability.”

187. What is initramfs?

> “initramfs is a temporary filesystem loaded into memory during boot.
> It helps the kernel mount the real root filesystem.”

188. What is GRUB?

> “GRUB is the bootloader.
> It allows you to choose which kernel to boot and pass boot parameters.”

189. How do you troubleshoot boot failures?

> “I check boot logs, kernel messages, and GRUB configuration.
> If needed, I boot into recovery mode.”

190. What is runlevel vs target?

> “Runlevels were used in older systems.
> Targets are the systemd replacement and are more flexible.”

191. What is secure boot?

> “Secure boot ensures only trusted software is loaded during boot.
> It helps protect against boot-level malware.”

192. What is kernel panic?

> “A kernel panic happens when the kernel encounters a fatal error.
> The system usually freezes or reboots immediately.”

193. How do you check kernel version?

> “I use `uname -r`.
> It’s useful when debugging driver or compatibility issues.”

194. What is a module in Linux?

> “A module is a piece of code that can be loaded into the kernel dynamically.
> It allows adding functionality without rebooting.”

195. How do you check loaded kernel modules?

> “I use `lsmod`.
> It helps when debugging hardware or driver-related issues.”

196. What is sysctl?

> “sysctl allows tuning kernel parameters at runtime.
> It’s commonly used for network and memory tuning.”

197. What is proc filesystem?

> “`/proc` is a virtual filesystem that exposes kernel and process information.
> It’s very useful for debugging.”

198. What is /sys filesystem?

> “`/sys` provides information about devices and drivers.
> It’s more structured compared to `/proc`.”

199. What is context switching?

> “Context switching happens when the CPU switches between processes or threads.
> Too much context switching usually means the system is overloaded.”

200. Why understanding Linux internals matters for DevOps?

> “Because production issues are rarely simple.
> Understanding internals helps you debug faster and avoid guesswork.”

201. Difference between process and thread (deep dive)

> "A process is isolated — it has its own memory and resources.
> Threads run inside a process and share memory.
> Threads are faster, but if one thread crashes badly, it can affect the whole process."

202. How do you troubleshoot high CPU usage in production?

> "I start with top or htop to identify the process.
> Then I check whether the CPU usage is expected or abnormal.
> If it's abnormal, I look at logs, recent changes, and traffic patterns."

203. How do you identify memory leaks in Linux?

> "I monitor a process over time.
> If memory usage keeps increasing and never drops, that's a strong sign of a leak.
> At that point, it usually needs application-level fixes."

204. Zombie vs orphan process (again, real-life explanation)

> "Zombie processes are dead but not cleaned up — that's bad code behavior.
> Orphan processes are still running and get adopted by systemd — that's normal."

205. What happens when a process is killed?

> "With a graceful kill, the process gets time to clean up.
> With a force kill, it stops immediately without cleanup.
> That's why SIGKILL should always be the last option."

206. What is CPU throttling?

> "CPU throttling happens when limits are applied, often in containers.
> The process isn't slow because of load — it's being restricted."

207. How do you check CPU limits?

> "On a VM, I check CPU cores and load.
> In containers, I check CPU limits defined in the container configuration."

208. What is memory overcommit?

> "Linux allows allocating more memory than physically available.
> It works until processes actually start using that memory — then OOM issues happen."

209. What is swappiness?

> "Swappiness controls how aggressively Linux uses swap.
> High swappiness means more swapping.
> In servers, I usually keep it low."

210. How do you tune swappiness?

> "Using sysctl.
> It's a runtime kernel tuning and doesn't require a reboot."

211. What is IO wait and why does it matter?

> "IO wait means the CPU is waiting for disk operations.
> High IO wait usually points to slow disks or heavy IO workloads."

212. How do you troubleshoot slow disk performance?

> "I check disk metrics using iostat and iotop.
> Then I identify which process is causing heavy disk usage."

213. Difference between load average and CPU usage

> "CPU usage shows how busy the CPU is.
> Load average shows how many processes are waiting.
> A system can have low CPU but high load if IO is slow."

214. What is context switching and why is too much bad?

> "Context switching happens when the CPU switches tasks.
> Too much switching means overhead and reduced performance."

215. How do you identify a slow Linux server?

> "I look at CPU, memory, disk IO, and network latency.
> If all look normal, I dig into application-level metrics."

216. What is `htop` useful for in production?

> "It gives a quick, visual understanding of system health.
> I can sort by CPU, memory, or process tree very easily."

217. What is `vmstat` used for?

> "vmstat gives a high-level view of memory, CPU, and processes.
> It's useful to quickly spot bottlenecks."

218. What is `iostat` used for?

> "iostat focuses on disk performance.
> I use it when IO wait is high."

219. What is `iotop` used for?

> "iotop shows which processes are doing heavy disk IO.
> Very useful during performance issues."

220. Performance troubleshooting mindset

> "I never look at just one metric.
> Performance issues are almost always a combination of CPU, memory, IO, and application behavior."

221. Explain Linux permission model in depth

> "Linux permissions are based on read, write, and execute, applied to owner, group, and others.
> This model decides who can access or modify files.
> Most real-world permission issues happen because ownership is wrong, not permissions."

222. What is SUID?

> "SUID allows a file to run with the owner's privileges, usually root.
> It's powerful but risky, so it should be used very carefully."

223. What is SGID?

> "SGID makes a file or directory inherit the group ownership.
> It's commonly used in shared directories where multiple users collaborate."

224. What is the sticky bit?

> "Sticky bit prevents users from deleting files they don't own in a shared directory.
> You'll often see it on /tmp."

225. Why are SUID/SGID risky?

> "Because if misused, they can allow privilege escalation.
> Any SUID binary must be audited regularly."

226. How do you find SUID files?

> "I use find / -perm -4000.
> It helps identify potential security risks."

227. How do you audit file permissions on a server?

> "I check critical directories like /etc, /var, and application paths.
> I also look for world-writable files, which are a big red flag."

228. How do you secure shared directories?

> "I use proper group ownership, SGID, and restrict write access.
> Sticky bit is useful in shared writable directories."

229. What is inode and why does it matter in security?

> "Inodes control access indirectly.
> If permissions on an inode are wrong, filenames don't matter.
> Security issues often hide at inode level."

230. Disk is full but `du` doesn't show large files — why?

> "Usually because deleted files are still open by running processes.
> Restarting the process releases the space."

231. How do you manage file ownership in production?

> "I always assign ownership to service users, not root.
> Running applications as root is risky."

232. What is `chown` vs `chmod`?

> "`chown` changes ownership.
> `chmod` changes permissions.
> Both are equally important."

233. How do you restrict file access to a specific group?

> "I change group ownership and set permissions accordingly.
> Groups are very powerful in Linux access control."

234. What is ACL (Access Control List)?

> "ACL allows more granular permissions beyond user, group, and others.
> It's useful when default permissions are not enough."

235. When do you use ACLs?

> "When multiple users need different access levels to the same file.
> It's common in shared enterprise environments."

236. How do you check ACL permissions?

> "I use getfacl.
> It shows all extended permissions."

237. How do you set ACL permissions?

> "Using setfacl.
> It gives fine-grained control but should be documented properly."

238. Why are world-writable files dangerous?

> "Any user can modify them.
> This can lead to security breaches or accidental corruption."

239. How do you secure application directories?

> "I restrict permissions, assign correct ownership, and avoid root execution.
> Security starts with least privilege."

240. Security mindset for filesystem management

> "Every permission change should be intentional.
> Loose permissions may fix issues quickly, but they create long-term risks."

241. Server rebooted unexpectedly — what do you check first?

> "First I check uptime to confirm when it rebooted.
> Then I check boot logs and kernel logs.
> Most unexpected reboots are due to OOM killer, kernel panic, or infra issues."

242. Application is running but service is not accessible

> "I check if the service is listening on the correct port.
> Then firewall rules, then network connectivity.
> If all that looks fine, I check application logs."

243. SSH login suddenly stopped working — possible reasons?

> "Disk full, permission issues in .ssh, SSH config changes, or firewall blocks.
> Disk full is surprisingly common."

244. System is slow but CPU and memory look normal

> "I check disk IO and network latency.
> Most people forget IO wait — that's usually the culprit."

245. Logs are not getting generated

> "First thing I check is disk space and permissions.
> If logs can't be written, services silently fail."

246. Disk usage alert triggered — how do you respond?

> "I identify which filesystem is full, then what's consuming space.
> Logs are usually the first suspect.
> After cleanup, I fix the root cause."

247. High CPU alert at 2 AM — what do you do?

> "I check which process is consuming CPU.
> If it's expected load, fine.
> If not, I investigate logs and recent changes before restarting anything."

248. Memory keeps increasing for a process

> "That's a memory leak sign.
> Restarting is a temporary fix; the real solution is application-level debugging."

249. Application crashed — what information do you collect?

> "Logs, timestamps, system resource usage, recent deployments.
> Without context, logs alone are misleading."

250. How do you troubleshoot a slow Linux server?

> "I look at CPU, memory, disk IO, and load average.
> Then network.
> If OS looks healthy, the issue is usually the application."

251. How do `top`, `htop`, `vmstat`, `iostat` help together?

> "top/htop show processes.
> vmstat shows system-level behavior.
> iostat shows disk performance.
> Together they give the full picture."

252. Disk full but `df` and `du` mismatch

> "Deleted files still open by processes.
> Restarting the process releases space."

253. What is ulimit and why is it important in production?

> "It prevents a single user or process from exhausting system resources.
> File descriptor limits are a common production issue."

254. How do you manage log rotation?

> "Using logrotate.
> Manual log cleanup is a red flag in production."

255. How do you secure a Linux server (summary)?

> "SSH keys only, firewall rules, updates, monitoring, least privilege.
> Security is layered, not one tool."

256. Difference between Bash and shell

> "Shell is the concept.
> Bash is one implementation.
> Bash has more features, so it's commonly used."

257. How do you pass arguments to a shell script?

> "Using positional parameters like $1, $2, $@.
> This makes scripts reusable."

258. Exit codes and error handling in Bash

> "Exit code 0 means success.
> Non-zero means failure.
> Automation relies heavily on this."

259. How do you write reusable Bash scripts?

> "By using functions, variables, and avoiding hardcoded values.
> Reusable scripts save a lot of time."

260. Functions in Bash — why use them?

> "They reduce duplication and improve readability.
> Large scripts without functions become unmaintainable."

261. set `-e`, `-x`, `-o pipefail` — why important?

> "They make scripts safer and easier to debug.
> Fail fast is better than silent failure."

262. How do you schedule scripts using cron?

> "Using crontab entries with proper logging and absolute paths."

263. Cron vs systemd timers

> "Cron is simple.
> Systemd timers are more powerful and better logged."

264. How do you run a script at system startup?

> "Using systemd service or timer.
> Startup scripts should be carefully tested."

265. How do you handle errors in Bash scripts?

> "Check exit codes, log errors, and stop execution if needed.
> Ignoring errors is dangerous."

266. What is `trap` in Bash?

> "It lets me clean up resources when a script exits or fails.
> Very useful for temp files."

267. How do you debug a Bash script?

> "I use `set -x`, add logging, and simplify the script until the issue is clear."

268. How do you secure sensitive data in scripts?

> "Never hardcode secrets.
> Use environment variables or secret managers."

269. How do you avoid hardcoding credentials?

> "By injecting secrets at runtime.
> If credentials are in Git, that's already a breach."

270. Bash vs Python — when do you choose Python?

> "When logic becomes complex or APIs are involved.
> Bash is great for orchestration, not heavy logic."

271. What is Linux and why is it used in DevOps?

> "Linux is stable, flexible, and scriptable.
> DevOps depends on automation, and Linux excels at that."

272. Linux vs Unix (practical view)

> "Linux is open-source and widely adopted.
> Unix is more vendor-specific."

273. What is root user?

> "Root has unrestricted access.
> That's why we avoid using it directly."

274. Root vs normal user

> "Root can break the system.
> Normal users are restricted for safety."

275. Explain Linux directory structure

> "/etc for configs, /var for logs, /usr for binaries, /opt for optional software.
> Knowing this helps troubleshooting."

276. chmod 755 vs 777

> "755 is controlled.
> 777 is dangerous and should be avoided."

277. df vs du

> "df shows filesystem usage.
> du shows directory usage."

278. ps vs top vs htop (again, interview short)

> "ps is snapshot, top is real-time, htop is interactive."

279. What is a daemon process?

> "Background service that runs continuously."

280. How do you check memory usage?

> "free, top, htop.
> For trends, monitoring tools."

281. How do you check disk usage?

> "df for overall, du for details."

282. How do you find large files?

> "Using du and find.
> Logs are usually the cause."

283. What is networking at OS level?

> "How Linux handles IPs, ports, sockets, routing, and firewalls."

284. DNS troubleshooting mindset

> "If DNS fails, apps look broken even when network is fine."

285. Port checks — why important?

> "Service can be running but not listening.
> Port checks catch that quickly."

286. Proxy configuration issues

> "Proxy misconfig breaks package installs and outbound calls."

287. Monitoring — why critical?

> "You can't fix what you don't see.
> Monitoring prevents surprises."

288. Log monitoring vs metrics

> "Logs explain *what happened*.
> Metrics show *how bad it is*."

289. What is observability from Linux perspective?

> "Logs, metrics, and system behavior together."

290. How do you prepare Linux servers for production?

> "Harden security, configure monitoring, test failure scenarios."

291. Most common Linux production mistake you've seen?

> "Disk full due to logs."

292. Most dangerous Linux command?

> "rm -rf / — especially as root."

293. Why do interviews focus so much on Linux?

> "Because everything in DevOps runs on Linux underneath."

294. How do you explain Linux concepts to juniors?

> "Using real examples, not theory.
> Linux makes sense once you see it in production."

295. What separates junior and senior Linux engineers?

> "Seniors think in failure scenarios, not commands."

296. Your approach during a production incident?

> "Stay calm, gather data, fix the impact first, then root cause."

297. What do you check before restarting a service?

> "Logs, dependencies, and whether restart will make it worse."

298. What's your Linux troubleshooting philosophy?

> "Start simple, verify assumptions, don't guess."

299. One Linux skill every DevOps engineer must master

> "Troubleshooting under pressure."